Less is More: Compact Clue Selection for Efficient Retrieval-Augmented Generation Reasoning

Qianchi Zhang*
School of Artificial Intelligence, Beihang University
Beijing, China
zhangqianchi@buaa.edu.cn

Hainan Zhang†
School of Artificial Intelligence, Beihang University
Beijing, China
zhanghainan@buaa.edu.cn

Liang Pang
Institute of Computing Technology, Chinese Academy of Sciences
Beijing, China
pangliang@ict.ac.cn

Yongxin Tong
School of Computer Science and Engineering, Beihang University
Beijing, China
yxtong@buaa.edu.cn

Hongwei Zheng
Beijing Academy of Blockchain and Edge Computing
Beijing, China
zhenghongwei2024@163.com

Zhiming Zheng
School of Artificial Intelligence, Beihang University
Beijing, China
zhengzhiming0130@163.com

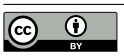
Abstract

Current RAG retrievers are designed primarily for human readers, emphasizing complete, readable, and coherent paragraphs. However, Large Language Models (LLMs) benefit more from precise, compact, and well-structured input, which enhances reasoning quality and efficiency. Existing methods rely on reranking or summarization to identify key sentences, but may introduce semantic breaks and unfaithfulness. Thus, efficiently extracting and organizing answer-relevant clues from large-scale documents while reducing LLM reasoning costs remains challenging in RAG systems. Inspired by Occam's razor, we frame LLM-centric retrieval as Min-Max optimization: maximizing the extraction of potential clues and reranking them for well-organization, while minimizing reasoning costs by truncating to the smallest sufficient set of clues. In this paper, we propose CompSelect, a compact clue selection mechanism for LLM-centric RAG, consisting of a clue extractor, a reranker, and a truncator. (1) The clue extractor first uses answer-containing sentences as fine-tuning targets, aiming to extract **sufficient** potential clues; (2) The reranker is trained to prioritize **effective** clues based on real LLM feedback; (3) The truncator uses the truncated text containing the minimum sufficient clues for answering the question as fine-tuning targets, thereby enabling **efficient** RAG reasoning. Experiments on three QA datasets demonstrate that CompSelect improves performance while reducing both total and online latency compared to a range of baseline methods. Further analysis also confirms its robustness to unreliable retrieval and generalization across different scenarios.

CCS Concepts

• **Information systems → Retrieval models and ranking**.

*This work was supported by Beijing Advanced Innovation Center for Future Blockchain and Privacy Computing.
†Corresponding author.



WWW '26, Dubai, United Arab Emirates.

ACM ISBN 979-8-4007-2307-0/2026/04
https://doi.org/10.1145/3774904.3792158

Keywords

Retrieval-Augmented Generation, Large Language Models, Information Retrieval

ACM Reference Format:
Qianchi Zhang, Hainan Zhang, Liang Pang, Yongxin Tong, Hongwei Zheng, and Zhiming Zheng. 2026. Less is More: Compact Clue Selection for Efficient Retrieval-Augmented Generation Reasoning. In *Proceedings of the ACM Web Conference 2026 (WWW '26), April 13–17, 2026, Dubai, United Arab Emirates.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3774904.3792158

1 Introduction

Retrieval-Augmented Generation (RAG) [7, 18, 51] has emerged as a promising paradigm to enhance Large Language Models (LLMs) with external knowledge for various tasks [2, 3, 10, 39]. However, the retrievers currently used in RAG are designed primarily for human readers, favoring long and coherent passages to enhance readability. This paradigm is less effective when the downstream consumer is an LLM [34], which typically performs better when provided with precise, compact, and well-structured inputs. As shown in Figure 1, we compare the Top-1 document, the full set of retrieved documents (approximately 5), and the answer-containing sentences (Oracle Clues) on the NQ dataset across different LLaMA3 model sizes. The results indicate that, regardless of whether it is a 1B model or the larger 70B model, structuring retrieved documents into a concise and efficient form (as Oracle Clues) consistently improves reasoning performance and reduces system latency. These findings highlight the need for compact clue selection mechanisms tailored to LLM reasoning in RAG systems.

Prior work has explored techniques such as reranking [16, 29, 42] and summarization [11, 53] to help RAG identify key sentences. The former reranks the sentences from retrieved documents based on metrics such as answer contribution or user preference [24, 54]. The latter identifies query-relevant sentences either through extractive selection or abstractive generation. Each approach has limitations: sentence-level reranking may disrupt the semantic structure of documents; extractive methods are limited in integrating dispersed information and capturing global context; and abstractive methods increase inference costs and may alter the original documents, thereby introducing issues of unfaithfulness. As shown in Figure 1,

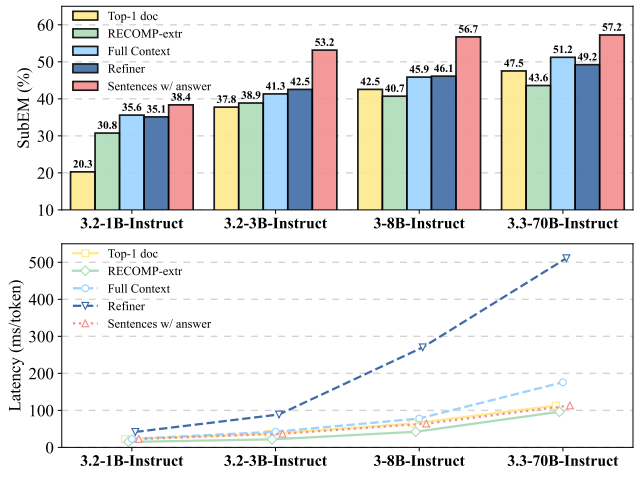


Figure 1: SubEM performance and latency on the NQ test set across different LLaMA3 model sizes, including the reranking method RECOMP-extr and the abstractive method Refiner.

the reranking method RECOMP-extr [42] primarily reduces reasoning latency but sacrifices QA performance, whereas the abstractive method Refiner [22] improves QA performance but increases system latency compared to baseline models. Therefore, efficiently extracting and organizing answer-relevant clues from large-scale documents while reducing LLM reasoning overhead remains an open challenge for RAG [19, 35].

Inspired by Occam's razor [5, 31], we frame LLM-centric retrieval as a MinMax optimization problem. We first leverage contextual information to identify potential clues, forming the maximal subset capable of answering the query. Next, we rerank these clues based on their completeness and relevance to the query, moving the most effective ones at the forefront. Finally, we truncate the set to retain only the essential clues, thereby minimizing the context required for RAG.

In this paper, we propose **CompSelect**, a compact clue selection mechanism for LLM-centric RAG, consisting of three modules: a clue extractor, a reranker, and a truncator. The extractor and reranker work together to provide sufficient and effective reasoning clues, while the truncator reduces reasoning cost. Specifically, (1) the clue extractor is fine-tuned using all sentences containing the answer and their semantically similar sentences based on K-Nearest Neighbors (KNN) clustering [8, 28], since RAG often requires richer contextual information for multi-hop reasoning. This ensures the extractor can capture **sufficient** potential clues. (2) The reranker is trained with real feedback from the LLM generator, labeling clues that lead to correct answers as positive samples and others as negative, enabling it to prioritize **effective** clues. (3) The truncator is fine-tuned using truncated texts that contain the minimum sufficient clues required to answer each question, and then applies this truncation to the reranked clues, thereby reducing reasoning tokens and improving the system's **efficiency**.

Experiments on NQ, TriviaQA, and HotpotQA datasets show that CompSelect consistently outperforms baselines while requiring significantly less context for inference on both LLaMA3 and Qwen3. In addition to performance gains, it proves robust to unreliable retrieval and generalizes well across scenarios. By aligning retrieval with the reasoning needs of LLMs, CompSelect provides a scalable, cost-efficient solution for web-scale RAG applications.

The innovations in this paper are as follows:

- We frame the LLM-centric compact clue selection mechanism as MinMax optimization, motivated by our finding that precise, compact, and well-organized inputs enhance both the LLM's reasoning performance and efficiency.
- We incorporate real feedback from the LLM: a KNN-based extractor gathers sufficient reasoning clues, a reranker organizes them into a coherent order, and a truncator improves reasoning density.
- Experiments on three datasets demonstrate that compact clue selection not only enhances inference performance and reduces system latency, but provides robustness against unreliable retrieval and supports generalization across tasks.

2 Related Work

Existing methods for identifying supporting content in RAG can be categorized as reranking or summarization, with summarization identifying query-relevant sentences via extractive selection or abstractive generation [20].

Reranking methods prioritize the most relevant content from retrieved documents. Classical methods like BM25 [33] rank documents based on term frequency, inverse document frequency, and document length. Neural rerankers [25, 26], such as BGE-Reranker [41] and RECOMP-extr [42], use dense embeddings or fine-tuned cross-encoders to select salient sentences. RichRAG [37] employs a generative list-wise ranker to produce and rank candidate documents, while Ke et al. [16] introduces a bridge mechanism to better connect retrievers and LLMs. CPC [23] ranks sentences with context-aware embeddings. Although reranking methods maintain low latency, they may harm RAG performance by disrupting the original logical structure of the document and can generate unfaithful clues.

Extractive methods aim to select key information from retrieved documents and can be further categorized into fragment selection and token pruning. Fragment selection methods, such as EXIT [11], use sentence-level classifiers to pick the most relevant sentences, achieving low latency but often relying on rigid selection criteria that may not adapt well to query complexity or document quality. Token-pruning methods, including LongLLMLingua [13] and Selective-Context [21], leverage LLMs to compute the informativeness of each token and remove low-information tokens, which may cause context fragmentation and higher latency. Other approaches, such as AdaComp [46], BlendFilter [36], ClueAnchor [1] and EviOmni [48], employ adaptive selection, knowledge blending, or reinforcement learning to improve reasoning efficiency and accuracy.

Abstractive methods generate query-relevant content via autoregressive generation. RECOMP-abs [42] uses a T5-based model [30], Refiner [22] hierarchically structures query-relevant content with larger LLMs, and BottleNeck [53] uses reinforcement learning with information bottleneck theory to generate task-relevant summaries.

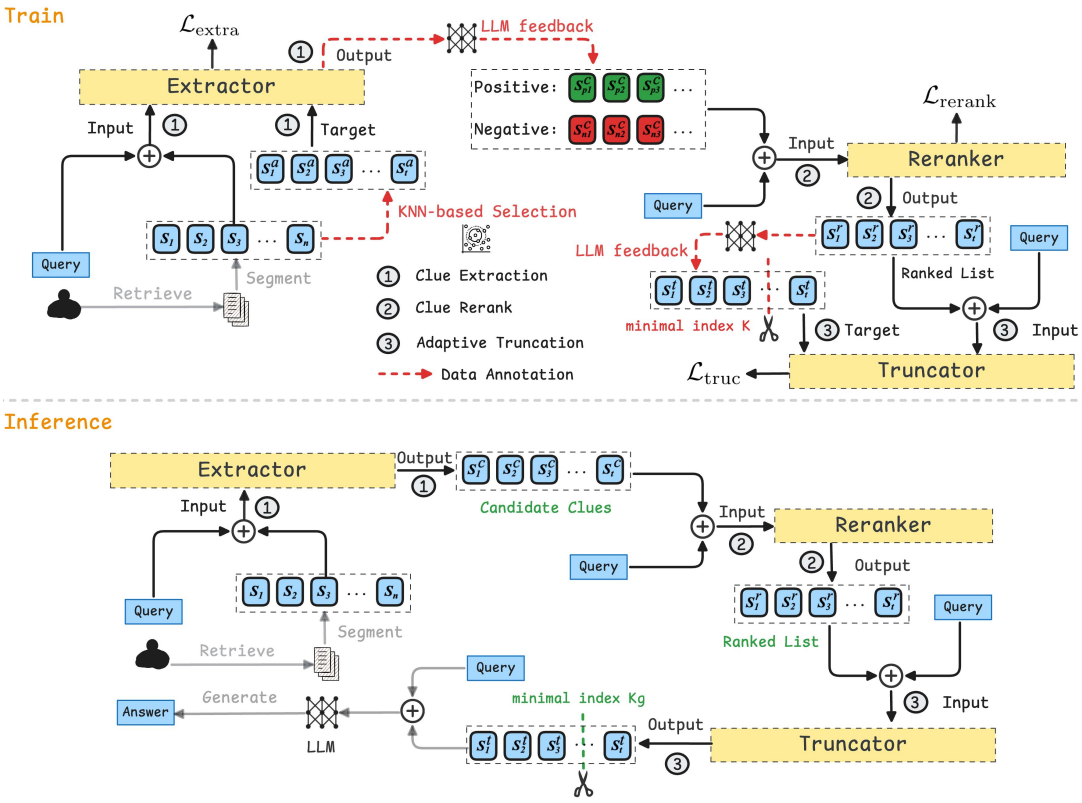


Figure 2: Architecture of CompSelect, consisting of three modules: a clue extractor, a reranker, and a truncator.

FILCO [40] trains a filtering model to dynamically select key sentences and jointly learns with the generator for end-to-end distillation. SEER [49] uses self-alignment to select evidence via expert evaluation, but still depends on domain-specific experts and can be sensitive to noisy retrieval. These approaches reorganize scattered information into concise outputs, and as a result of their autoregressive generation, they typically incur higher latency.

In contrast, our work maximizes sufficiency, prioritizes effectiveness, and minimizes reasoning costs, enhancing both efficiency and accuracy in LLM-centric RAG.

3 Problem Formulation

Given a query q and its retrieved document set $\mathcal{D} = \{d_1, \ldots, d_n\}$, where each document d_i consists of a set of sentences $\mathcal{S}_i = \{s_1^i, \ldots, s_{n_i}^i\}$, with n_i denoting the number of sentences in d_i. The objective of our task is to identify an optimal subset $\mathcal{S}^* \subseteq \bigcup_{i=1}^n \mathcal{S}_i$ such that the language model f_θ generates the correct answer y for the query q, while minimizing the subset size. The optimal subset $\mathcal{S}^*$ can be determined by the following MinMax optimization:

$$\mathcal{S}^* = \arg \min_{\mathcal{S}' \subseteq \mathcal{S}_{\max}} |\mathcal{S}'|, \tag{1}$$

$$\mathcal{S}_{\max} = \arg \max_{\mathcal{S} \subseteq \bigcup_{i=1}^n \mathcal{S}_i} \log P_\theta(y \mid \mathcal{S}, q), \tag{2}$$

where $\mathcal{S}'$ is the minimal subset of $\mathcal{S}_{\max}$ that still allows the model f_θ to generate the correct answer and $|\mathcal{S}'|$ denotes the number of sentences in $\mathcal{S}'$. The selection of $\mathcal{S}^*$ should dynamically adapt to the real feedback of a RAG system to balance informativeness and conciseness.

4 Methodology

In this section, we introduce CompSelect, a three-stage framework for selecting and prioritizing relevant information in RAG, as illustrated in Figure 2. CompSelect consists of three modules: a clue extractor, a clue reranker, and an adaptive truncator. The clue extractor identifies candidate answer clues from multiple documents, reducing the search space while retaining relevant information. The reranker then prioritizes the most relevant clues using a pairwise loss strategy. Finally, the truncator adaptively selects the minimal necessary context for each query, thereby improving inference efficiency and answer accuracy [1].

4.1 Clue Extractor

The goal of the clue extractor is to identify potential answer clues from multiple documents and construct a smaller query-relevant candidate set to reduce the search space. We compare the performance of answer-containing sentences and the original retrieved documents in downstream tasks, as shown in Figure 1. The results indicate that filtering out low-value information from documents benefits RAG reasoning. Although not all answer-containing sentences are query-relevant, they approximate the maximal subset

[1] Our code is available at https://github.com/zqc1023/CompSelect.

capable of addressing user queries, and can serve as the optimization target for the clue extractor.

To guide the extraction of informative sentences, we first identify sentences from the retrieved documents that contain the ground-truth answer as targets for extraction. Given a query q and the ground-truth answer y, we denote by $\mathcal{S} = \{s_1, \ldots, s_n\}$ the set of sentences appearing in the retrieved documents, with sentences preserving their original order within and across documents. The answer-containing sentences are defined as:

$$\mathcal{S}^a = \{s_j | y \sqsubseteq s_j, s_j \in \mathcal{S}\}, \tag{3}$$

where $y \sqsubseteq s_j$ indicates that y is a substring of s_j.

Next, we fine-tune an LLM as the clue *Extractor* to generate answer-containing sentences $\mathcal{S}^a$ based on the query q and the retrieved sentences $\mathcal{S}$ with a specific prompt (see Appendix A.1). The loss function of *Extractor* model is defined as:

$$\mathcal{L}_{\text{extra}} = -\frac{1}{N}\sum_{i=1}^{N} \log P_\theta(\mathcal{S}_i^a \mid q_i, \mathcal{S}_i). \tag{4}$$

Finally, the clue extractor is capable of generating the potential candidate clues based on the user query and retrieved sentences during inference:

$$\mathcal{S}^c = Extractor(q, \mathcal{S}). \tag{5}$$

This procedure enables the RAG system to focus on informative content, thereby improving efficiency and answer accuracy.

Table 1: SubEM (%) comparison of different strategies on NQ, TriviaQA, and HotpotQA test set. We use LLaMA3-8B-Instruct as the generator. ϵ shows the KNN threshold and higher values introduce more contextual sentences.

Method	NQ	TriviaQA	HotpotQA
Full Content	45.87	67.08	25.66
Sentences *w/* answer	**56.73**	71.52	30.46
KNN-based Extraction ($\epsilon = 0.15$)	56.41	**71.98**	**30.63**

4.1.1 ***KNN-based Extraction***. We further augment the clue extractor with a KNN-based strategy. As shown in Table 1, selecting answer-containing sentences substantially improves performance on single-hop QA datasets such as NQ, raising the SubEM score from 45.87 to 56.73. This highlights the benefit of focusing on relevant context for simple queries. For multi-hop or more complex QA datasets (e.g., TriviaQA and HotpotQA), incorporating answer-containing sentences also leads to performance gains. Moreover, further retrieving semantically similar sentences using a KNN-based strategy yields additional improvements, indicating that extra contextual information beyond strictly answer-containing sentences is beneficial for multi-hop reasoning and complex query understanding. Based on these observations, we propose a KNN-based extraction strategy. For simple questions, we select answer-containing sentences as optimization targets to provide key information. For complex questions, we additionally retrieve semantically similar sentences to supplement context. By using both answer-containing and KNN-based similar sentences as optimization targets, the extractor adapts to queries of varying complexity, improving reasoning accuracy while maintaining a concise candidate set.

4.2 Clue Reranker

The clue extractor often selects sentences with multiple relevant clues of varying importance, necessitating reranking. We address this by training a reranker with pairwise loss to prioritize the most relevant sentences.

4.2.1 ***Training***. We use real downstream generator feedback from the RAG system to annotate the training data for the reranker, as QA performance on complex questions heavily depends on the characteristics of the generation module. First, we pair each of the extracted clue sentences $s_j^c \in \mathcal{S}^c$ with the query q as (s_j^c, q), where sentence s_j^c that enables the downstream generation module to produce the correct answer for q is considered as positive sample s_{pos}, while other sentences are treated as negative samples s_{neg}[2]. The *Reranker* aims to minimize the following pairwise loss function [15] to improve relevance ranking:

$$\mathcal{L}_{\text{rerank}} = -\frac{1}{N}\sum_{i=1}^{N} \log \frac{e^{\text{sim}(q_i, s^i_{\text{pos}})}}{e^{\text{sim}(q_i, s^i_{\text{pos}})} + e^{\text{sim}(q_i, s^i_{\text{neg}})}}, \tag{6}$$

where $\text{sim}(q, *)$ represents the semantic similarity between the query q and the sentence $*$ by *Reranker* model. Minimizing this loss function enables the *Reranker* model to effectively identify and prioritize the most relevant clues.

4.2.2 ***Inference***. Given the query q and the extracted sentences $\mathcal{S}^c$, the *Reranker* model calculates the relevance score between every sentence $s_j^c \in \mathcal{S}^c$ and query q. The reranked clue set is defined as:

$$\mathcal{S}^r = Reranker(q, \mathcal{S}^c). \tag{7}$$

4.3 Adaptive Truncator

The adaptive truncator is designed to identify the minimal necessary clues based on the complexity of the question and the documents, ensuring sufficient clues for accurate answer generation.

4.3.1 ***Training***. To determine the optimal clue subset $\mathcal{S}^t$ for each query q, we perform data annotation based on the reranked answer clues $\mathcal{S}^r$ obtained from the previous reranking step. Given a query q and its reranked clues $\mathcal{S}^r = \{s_1^r, \ldots, s_n^r\}$, the objective is to identify the smallest subset $\mathcal{S}^t$ such that the RAG system's generation model M can generate the correct answer y based on q and $\mathcal{S}^t$. We define $D_k = \{s_1^r, \ldots, s_k^r\}$, where $1 \le k \le n$. The performance on each subset D_k is evaluated by checking if the generation model's output $M(q, D_k)$ matches the ground truth y. The correctness condition is defined as:

$$\text{Correct}(q, D_k) = \begin{cases} 1, & \text{if } M(q, D_k) = y \\ 0, & \text{otherwise} \end{cases}. \tag{8}$$

Since the reranker cannot guarantee that the most relevant sentences are always ranked first, especially for complex questions, we iterate over the subsets from largest to smallest, starting with D_n and continuing to D_1. The optimal subset $\mathcal{S}^t$ is the smallest subset

[2]If no candidate clues can generate the correct answer, or if all samples can generate the correct answer, the sample will be removed from the annotated data.

that generates the correct answer:

$$\mathcal{S}^t = \{s_1^r, \ldots, s_K^r\}, \tag{9}$$

$$K = \arg\min_k \{k \mid \text{Correct}(q, D_k) = 1\}. \tag{10}$$

If the RAG system cannot generate a correct answer from any subset, then $\mathcal{S}^t = \emptyset$, indicating no subset suffices. This method ensures the use of minimal necessary context $\mathcal{S}^t$.

During the model training stage, we fine-tune an LLM based on the data annotations. The *Truncator* is trained to predict the minimal subset of reranked clues $\mathcal{S}^t \subseteq \mathcal{S}^r$ that suffices to answer the query:

$$\mathcal{L}_{\text{truc}} = -\frac{1}{N} \sum_{i=1}^{N} \log P_\theta(\mathcal{S}_i^t | q_i, \mathcal{S}_i^r). \tag{11}$$

4.3.2 ***Inference***. During inference, given a new query q and its reranked sentences $\mathcal{S}^r$, the *Truncator* truncates $\mathcal{S}^r$ to the minimal sufficient clue set $\mathcal{S}^t = \{s_1^r, \ldots, s_{K_g}^r\}$, where K_g represents the number of sentences needed to answer the query. The truncation process uses the prompt provided in Appendix A.2. Finally, the RAG generation module concatenates the query q with the filtered answer clues $\mathcal{S}^t$ using the following prompt to reason the answer:

```
<system>
You are a helpful, respectful, and honest assistant. Answer
the question with a couple of words using the provided
documents. For example: Question: What is the capital of
France? Output: Paris.
</system>
<user>
Question: {query}
Documents:
Doc1: {Document 1}
Doc2: {Document 2}
.....
</user>
```

5 Experiments

5.1 Experimental Setup

5.1.1 ***Datasets***. We evaluate our method on three QA datasets, including (1) Open-Domain QA, represented by NaturalQuestions (NQ) [17] and TriviaQA [14]; (2) Multi-Hop QA, represented by HotpotQA [45]. Table 11 in Appendix B.1 illustrates the statistics of these datasets.

5.1.2 ***Evaluation Metrics***. Since answer style mismatch may cause additional variance, we follow prior work [38, 43, 52] and adopt Substring Exact Match (**SubEM**) and **F1** for evaluation. SubEM checks whether the gold answer appears as a substring in the prediction, while F1 measures token-level overlap with the reference. For efficiency, we report the compression ratio (**CR**), defined as the ratio of original to compressed context length. We also measure **Total Latency**, including offline preprocessing and online answer generation, and **Online Latency**, the time from user query submission to answer generation using the preprocessed results, which can provide a more comprehensive assessment of practical efficiency.

5.1.3 ***Baselines***. We consider four baseline strategies:

Vanilla Methods: (i) ***Naive Generation***, which relies solely on the generator's parametric knowledge; (ii) ***Full Content***, which concatenates all the retrieved documents as input.

Reranking Methods: (i) ***BM25*** [33], a classic lexical matching method that scores and ranks documents based on term frequency, inverse document frequency, and document length normalization; (ii) ***BGE-reranker*** [41], a neural reranker that computes dense embeddings for queries and documents, ranking documents according to semantic similarity in the embedding space; (iii) ***RECOMP-extr*** [42], which employs a fine-tuned cross-encoder to select salient sentences through dense retrieval.

Extractive Methods: (i) ***LongLLMLingua*** [13], which prunes irrelevant tokens in long contexts via a dynamic programming algorithm guided by question-aware perplexity scores; (ii) **Selective-Context** [21], which uses self-information estimated by an external LLM to prune redundant words; (iii) ***EXIT*** [11], which compresses retrieved documents by applying sentence-level relevance classification to select and reassemble only the most relevant sentences for answering queries.

Abstractive Methods: (i) ***RECOMP-abs*** [42], which uses a T5-based model to perform abstractive summarization, compressing documents into shorter token sequences through autoregressive generation; (ii) ***Refiner*** [22], which leverages large LLMs to extract and structure query-relevant content from retrieved documents, producing a hierarchical output based on intrinsic document knowledge; (iii) ***BottleNeck*** [53], which employs reinforcement learning and information bottleneck theory to improve both filtering and generation.

5.1.4 ***Implementation Details***. We use LLaMA3-8B-Instruct [6], Qwen3-14B [44], and LLaMA3.3-70B-Instruct [6] (presented in Appendix C) as the generators, covering medium, large, and ultra-large LLMs. To ensure high coverage and quality of retrieved information [4], we follow prior work [22, 42, 47, 53] and utilize the adversarial Dense Passage Retriever (DPR) [15] to retrieve Top-5 passages from the full Wikipedia passages for each query. All baselines and our method are conducted using the same test sets and retrieval corpus, which guarantees consistency between baseline methods and our approach. To further reduce computational cost and latency, our clue extractor and clue truncator are based on LLaMA3.2-3B-Instruct [6] and we train the model using the LoRA method [9] within the LLaMAFactory framework [50]. For the clue reranker, we implement Sentence-BERT [32] using distilbert-base-uncased. More details are provided in Appendix B.2.

5.2 Main Results

The comparison results for the three datasets are shown in Table 2. The results indicate the following: (i) **RAG improves downstream task performance across all datasets.** Incorporating retrieved documents consistently boosts answer accuracy compared with using the generator alone; (ii) **Compact clue selection**

Table 2: Experimental results (%) on three datasets using LLaMA3-8B-Instruct and Qwen3-14B as the generators. The retriever is DPR. All baselines and our method are conducted using the same test sets and retrieval corpus.

Method	NQ			TriviaQA			HotpotQA		
	SubEM	F1	CR	SubEM	F1	CR	SubEM	F1	CR
LLaMA3-8B-Instruct									
Naive Generation	25.18	29.11	-	55.92	58.95	-	21.39	22.87	-
Full Content	45.87	47.86	1×	67.08	68.61	1×	25.66	28.22	1×
BM25	38.97	41.21	4.1×	53.17	56.73	4.3×	21.85	23.14	4.4×
Bge-reranker	41.22	45.56	4.1×	57.93	60.43	4.3×	22.73	24.01	4.5×
RECOMP-extr	40.71	45.35	11.97×	63.73	66.46	10.91×	24.39	26.54	8.33 ×
LongLLMLingua	41.85	45.74	4.56×	64.92	66.87	4.18×	23.74	26.19	4.45×
Selective-Context	43.84	46.33	2.6×	61.53	61.02	2.7×	24.28	26.51	2.7×
EXIT	41.19	45.44	14.16×	64.03	66.46	12.78×	24.16	25.80	**15.43**×
RECOMP-abs	43.11	46.16	11.12×	61.89	61.35	11.25×	23.55	25.29	7.91×
Refiner	46.12	48.37	10.97×	65.97	67.64	12.63×	24.72	26.78	7.65×
BottleNeck	45.32	47.21	14.32×	66.98	68.31	**13.17**×	25.71	28.13	13.21×
Ours ($\epsilon = 0$)	**46.98**	**49.45**	**14.95**×	**68.29**	**69.72**	12.54×	**26.14**	**28.43**	13.56×
Qwen3-14B									
Naive Generation	27.92	29.01	-	56.56	57.71	-	23.59	29.51	-
Full Content	51.52	48.55	1×	72.67	72.10	1×	30.05	34.32	1×
BM25	40.88	45.52	4.1×	61.15	60.91	4.3×	24.08	26.31	4.4×
Bge-reranker	42.95	46.15	4.1×	63.12	66.57	4.3×	24.73	26.79	4.5×
RECOMP-extr	43.29	46.25	11.97×	62.94	63.86	10.91×	25.77	29.83	8.33×
LongLLMLingua	44.79	46.93	4.56×	68.73	68.39	4.18×	26.43	30.21	4.45×
Selective-Context	49.31	47.22	2.6×	65.59	67.11	2.7×	26.07	30.05	2.7×
EXIT	42.23	45.77	14.16×	63.78	64.85	12.78×	28.16	33.89	**15.43**×
RECOMP-abs	45.75	47.56	11.12×	65.34	66.93	11.25×	27.45	31.19	7.91×
Refiner	51.14	48.16	10.97×	71.18	70.55	12.63×	30.12	34.39	7.65×
BottleNeck	50.72	47.78	14.32×	72.36	71.87	13.17×	29.64	33.72	13.21×
Ours ($\epsilon = 0$)	**52.33**	**49.32**	**16.18**×	**72.93**	**72.61**	**13.35**×	**30.67**	**34.61**	14.17×

improves information utilization. Compared with Full Content, our method significantly compresses the context (high CR) while maintaining or improving SubEM and F1, effectively reducing redundant information and retaining critical clues for accurate reasoning; (iii) **CompSelect consistently achieves the best overall performance.** Across different datasets and generators, our method attains the highest SubEM and F1 scores, highlighting its effectiveness in extracting key information and generating accurate answers; (iv) **CompSelect maintains robust performance and generalizes well across datasets and models.** Across multiple datasets and generators, our method consistently outperforms the baselines, indicating stable performance and strong generalization in diverse QA scenarios.

5.3 Ablation Study

5.3.1 ***Overall.*** To explore the impact of different components of CompSelect, we use LLaMA3-8B-Instruct as the base LLM and introduce the following variants for ablation study: 1) *w/o clue extractor.* This variant directly uses LLaMA3.2-3B-Instruct without fine-tuning for clue extraction. 2) *w/o clue reranker.* This variant skips reranking and retains the original sentence order. 3) *w/o adaptive truncator.* This variant disables adaptive truncation of the reranked clues. As shown in Table 3, removing any single component leads to a drop in SubEM, indicating that each module contributes significantly to the overall performance.

Table 3: Ablation study on NQ, TriviaQA and HotpotQA test set. We use LLaMA3-8B-Instruct as the generator and SubEM (%) for evaluation.

Method	NQ	TriviaQA	HotpotQA
CompSelect (Ours)	**46.98**	**68.29**	**26.14**
w/o clue extractor	45.83	66.71	25.12
w/o clue reranker	46.55	67.95	25.84
w/o adaptive truncator	46.72	67.89	25.91

5.3.2 ***Direct vs. Fine-tuned Extraction.*** We analyze the effect of fine-tuning on clue extraction by comparing two approaches: 1) *Direct Extraction*, which uses the extractor without fine-tuning to extract clue sentences from retrieved documents based on the given prompt (see Appendix A.1); 2) *Fine-tuned Extraction*, which uses fine-tuned extractor to select answer-containing sentences. As shown in Table 4, Fine-tuned Extraction consistently outperforms Direct Extraction by leveraging task-specific knowledge to identify more relevant sentences.

5.3.3 ***Threshold of KNN-Based Extraction.*** We assess the KNN-based extraction method by varying the threshold, which controls the cosine similarity to answer-containing sentences. A threshold of 0 selects only the answer sentences, while higher thresholds allow semantically similar sentences to expand the context with

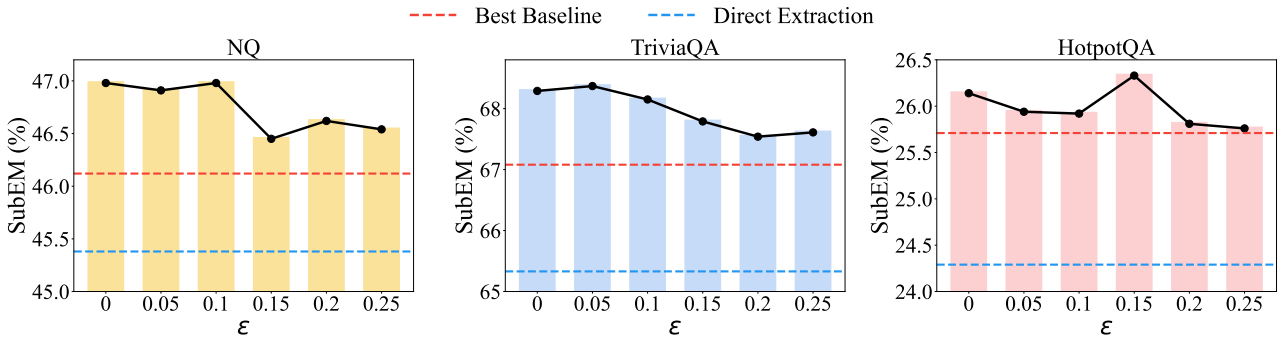


Figure 3: An illustration of clue extraction performance using LLaMA3-8B-Instruct as generator on NQ, TriviaQA, and HotpotQA test sets. The x-axis shows the KNN threshold and higher values introduce more contextual sentences.

Table 4: Performance (%) comparison between Direct Extraction and Fine-tuned Extraction across three datasets. We use LLaMA3-8B-Instruct as the generator.

Dataset	Method	SubEM	F1
NQ	Direct Extraction	45.38	47.27
	Fine-tuned Extraction	**46.43**	**49.45**
TriviaQA	Direct Extraction	65.33	68.28
	Fine-tuned Extraction	**67.54**	**69.72**
HotpotQA	Direct Extraction	24.29	25.74
	Fine-tuned Extraction	**25.84**	**28.50**

additional relevant information. As shown in Figure 3, we compare the model's performance at different threshold values. For simple questions such as NQ, the KNN extraction strategy does not improve performance, as answers can typically be obtained directly from sentences containing the answers. For more complex questions such as HotpotQA and TriviaQA, the KNN strategy improves performance at lower thresholds but declines at higher thresholds due to increased noise. Notably, KNN-based extraction serves a supplementary role in the overall framework. While the threshold setting impacts CompSelect's performance, it does not alter the experimental finding that CompSelect outperforms the best baseline.

5.3.4 ***Random vs. Adaptive Truncation.*** To validate the effectiveness of our adaptive truncation strategy, we compare it with random truncation. As shown in Table 5, our adaptive truncation consistently outperforms random truncation in both SubEM and F1 scores. This improvement arises because adaptive truncation dynamically selects the most relevant context based on real-time feedback from the downstream generator, retaining sentences that are both highly informative and directly pertinent to the query. By doing so, it preserves critical information while reducing unnecessary content, enhancing the model's answer accuracy and overall reasoning capability. In contrast, random truncation does not account for the importance of individual sentences and may discard key clues, leading to degraded performance.

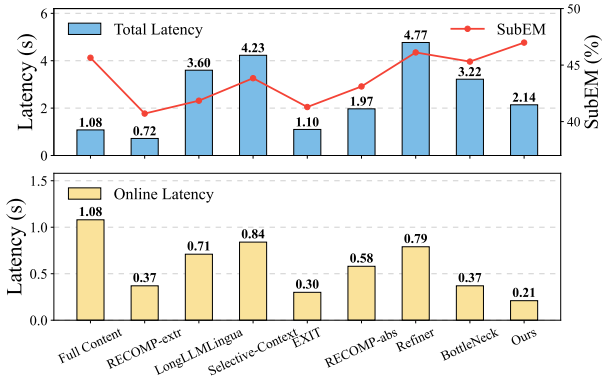


Figure 4: Latency comparison across baselines and our method using LLaMA3-8B-Instruct. The top shows Total Latency along with SubEM performance, while the bottom shows Online Latency. Experiments were conducted on two NVIDIA RTX PRO 6000 GPUs.

Table 5: Performance (%) comparison between Random Truncation and Adaptive Truncation across three datasets. We use Llama3-8B-Instruct as the generator. We report SubEM and F1 for evaluation.

Dataset	Method	SubEM	F1
NQ	Random	46.35	48.44
	Adaptive Truncation	**46.98**	**49.45**
TriviaQA	Random	67.71	69.13
	Adaptive Truncation	**68.29**	**69.72**
HotpotQA	Random	25.73	28.17
	Adaptive Truncation	**26.14**	**28.43**

5.4 System Latency Evaluation

To evaluate the overall efficiency of each method, we measured the average total latency (including offline preprocessing and online answer generation) and online latency (the time to answer

Table 6: Cascading errors analysis of extractor (Recall-1), reranker (Hit-2@1) and truncator (Recall-3) on three datasets, with ϵ set to 0.

Dataset	Recall-1 (%)	Hit-2@1 (%)	Recall-3 (%)
NQ	80.71	78.35	79.46
TriviaQA	92.33	84.46	86.07
HotpotQA	84.12	77.59	80.16

Table 7: Results under unreliable retrieval. We use LLaMA3-8B-Instruct as the generator and F1 (%) for evaluation. The best results are in bold and the second are underlined.

Method	NQ	HotpotQA
RECOMP-extr	14.93	15.65
LongLLMLingua	14.77	15.77
Selective-Context	**15.42**	<u>16.35</u>
EXIT	14.59	15.25
RECOMP-abs	13.74	15.08
Refiner	15.13	16.26
BottleNeck	14.58	16.18
Ours (*w/o Truncator*)	14.32	16.12
Ours (*w/ Truncator*)	<u>15.36</u>	**16.87**

generation using the preprocessed results) for processing a single sample on the NQ test set, along with the corresponding SubEM scores. This measurement setup provides a more comprehensive assessment of practical efficiency. As shown in Figure 4, RECOMP-extr exhibits the lowest total latency but relatively lower SubEM, whereas our method achieves the highest SubEM while maintaining a relatively low total latency and online latency. This indicates that with lightweight reranker and truncator design, our method can improve accuracy without significantly increasing latency, thereby achieving a better trade-off between efficiency and accuracy.

5.5 Robustness Analysis

5.5.1 ***Cascading Errors Resilience Analysis.*** To quantify cascading errors, we conduct experiments on the three test sets in which the retrieved documents contain the gold-standard answers. Specifically, **Recall-1** evaluates whether the extractor successfully recalls the gold-standard answer, **Hit-2@1** measures the probability that the reranker ranks the gold-standard answer as Top-1, and **Recall-3** assesses whether the truncator continues to retain the gold-standard answer after truncation. These metrics provide a systematic means to analyze how errors propagate across the sequential components of the system. As shown in Table 6, CompSelect consistently maintains a low error rate across all modules, indicating that cascading errors are effectively controlled and kept within a manageable range, which demonstrates the robustness of the framework in preserving critical information throughout the processing pipeline.

5.5.2 ***Performance under Unreliable Retrieval.*** The truncator not only shortens context but also helps filter out unreliable content. We conduct experiments on samples from the NQ and HotpotQA test sets without gold-standard answers to simulate unreliable retrieval. As shown in Table 7, CompSelect's truncator effectively suppresses noise and prevents answer degradation. This is because CompSelect is trained to allow empty outputs, whereas baseline models perform poorly under unreliable retrieval conditions, as they must choose an output answer clue.

Table 8: Comparison of cross-task generalization ability across different baselines. We use ROUGE-1, ROUGE-2, and ROUGE-L for evaluation.

Method	ROUGE-1	ROUGE-2	ROUGE-L
Full Content	0.3769	0.1546	0.2234
RECOMP-extr	0.3051	0.1022	0.1668
LongLLMLingua	0.4082	0.1775	0.2369
Selective-Context	0.3927	0.1611	0.2275
EXIT	0.3842	0.1573	0.2219
RECOMP-abs	0.3795	0.1498	0.2146
Refiner	0.4018	0.1729	0.2336
BottleNeck	0.3708	0.1529	0.2158
Ours	**0.4123**	**0.1798**	**0.2512**

5.6 Cross-Task Generalization

To further evaluate the generalization capability of CompSelect, we perform inference on 1,200 randomly sampled instances from a Conversational Multi-Doc QA dataset [3], with the model trained solely on HotpotQA dataset. This setup allows us to examine whether the model can adapt to a new conversational QA scenario without direct exposure during training. As shown in Table 8, CompSelect consistently achieves strong performance in this unseen dataset, highlighting its robustness and ability to generalize across diverse QA tasks.

6 Conclusion

In this work, we propose CompSelect, a compact clue selection mechanism for LLM-centric RAG that efficiently extracts, organizes, and truncates answer-relevant information from large-scale documents. By framing retrieval as a MinMax optimization, its three modules, the clue extractor, the reranker, and the truncator, work together to enhance reasoning quality and efficiency. Experiments on three QA datasets show that CompSelect improves performance and reduces latency, while remaining robust to unreliable retrieval and generalizing well across scenarios. Future work could explore integrating it with generative retrieval, allowing models to generate document indices, thereby avoiding costly online processing and further reducing end-to-end latency.

Acknowledgements

This work was funded by the National Natural Science Foundation of China (NSFC) under Grants No.U25B2070 and No. 62406013, the Beijing Advanced Innovation Center Funds for Future Blockchain and Privacy Computing(GJJ-24-034), and the Fundamental Research Funds for the Central Universities.

[3] https://sites.google.com/view/wsdm24-docqa

References

[1] Hao Chen, Yukun Yan, Sen Mei, Wanxiang Che, Zhenghao Liu, Qi Shi, Xinze Li, Yuchun Fan, Pengcheng Huang, Qiushi Xiong, Zhiyuan Liu, and Maosong Sun. 2025. ClueAnchor: Clue-Anchored Knowledge Reasoning Exploration and Optimization for Retrieval-Augmented Generation. In *Findings of the Association for Computational Linguistics: EMNLP 2025*. 19258–19278.

[2] Jinwen Chen, Hainan Zhang, Liang Pang, Yongxin Tong, Haibo Zhou, Yuan Zhan, Wei Lin, and Zhiming Zheng. 2025. Privacy-Preserving Reasoning with Knowledge-Distilled Parametric Retrieval Augmented Generation. *arXiv preprint arXiv:2509.01088* (2025).

[3] Jiangui Chen, Ruqing Zhang, Jiafeng Guo, Yixing Fan, and Xueqi Cheng. 2022. GERE: Generative evidence retrieval for fact verification. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2184–2189.

[4] Florin Cuconasu, Giovanni Trappolini, Federico Siciliano, Simone Filice, Cesare Campagnano, Yoelle Maarek, Nicola Tonellotto, and Fabrizio Silvestri. 2024. The power of noise: Redefining retrieval for rag systems. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 719–729.

[5] Guillaume d'Ockham. 1938. *Ockham Studies and Selections*. Open Court Publishing-Company.

[6] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv e-prints* (2024), arXiv–2407.

[7] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997* (2023).

[8] Gongde Guo, Hui Wang, David Bell, Yaxin Bi, and Kieran Greer. 2003. KNN model-based approach in classification. In *On The Move to Meaningful Internet Systems 2003: CoopIS, DOA, and ODBASE: OTM Confederated International Conferences, CoopIS, DOA, and ODBASE 2003, Catania, Sicily, Italy, November 3-7, 2003. Proceedings*. Springer, 986–996.

[9] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685* (2021).

[10] Qiushi Huang, Shuai Fu, Xubo Liu, Wenwu Wang, Tom Ko, Yu Zhang, and Lilian Tang. 2023. Learning Retrieval Augmentation for Personalized Dialogue Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Singapore, 2523–2540.

[11] Taeho Hwang, Sukmin Cho, Soyeong Jeong, Hoyun Song, SeungYoon Han, and Jong C. Park. 2025. EXIT: Context-Aware Extractive Compression for Enhancing Retrieval-Augmented Generation. In *Findings of the Association for Computational Linguistics: ACL 2025*. 4895–4924.

[12] Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. 2021. Unsupervised dense information retrieval with contrastive learning. *arXiv preprint arXiv:2112.09118* (2021).

[13] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024. LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 1658–1677.

[14] Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. 2017. TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Regina Barzilay and Min-Yen Kan (Eds.). Association for Computational Linguistics, Vancouver, Canada, 1601–1611. doi:10.18653/v1/P17-1147

[15] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, Online, 6769–6781. doi:10.18653/v1/2020.emnlp-main.550

[16] Zixuan Ke, Weize Kong, Cheng Li, Mingyang Zhang, Qiaozhu Mei, and Michael Bendersky. 2024. Bridging the Preference Gap between Retrievers and LLMs. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 10438–10451. doi:10.18653/v1/2024.acl-long.562

[17] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. 2019. Natural Questions: A Benchmark for Question Answering Research. *Transactions of the Association for Computational Linguistics* 7 (2019), 452–466.

[18] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems* 33 (2020), 9459–9474.

[19] Xiaoxi Li, Wenxiang Jiao, Jiarui Jin, Guanting Dong, Jiajie Jin, Yinuo Wang, Hao Wang, Yutao Zhu, Ji-Rong Wen, Yuan Lu, and Zhicheng Dou. 2025. DeepAgent: A General Reasoning Agent with Scalable Toolsets. arXiv:2510.21618 [cs.AI] https://arxiv.org/abs/2510.21618

[20] Xiangci Li and Jessica Ouyang. 2024. How Does Knowledge Selection Help Retrieval Augmented Generation? *arXiv preprint arXiv:2410.13258* (2024).

[21] Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. 2023. Compressing Context to Enhance Inference Efficiency of Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 6342–6353.

[22] Zhonghao Li, Xuming Hu, Aiwei Liu, Kening Zheng, Sirui Huang, and Hui Xiong. 2024. *Refiner*: Restructure Retrieved Content Efficiently to Advance Question-Answering Capabilities. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 8548–8572.

[23] Barys Liskavets, Maxim Ushakov, Shuvendu Roy, Mark Klibanov, Ali Etemad, and Shane K Luke. 2025. Prompt compression with context-aware sentence encoding for fast and improved llm inference. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 24595–24604.

[24] Jiaxin Mao, Yiqun Liu, Ke Zhou, Jian-Yun Nie, Jingtao Song, Min Zhang, Shaoping Ma, Jiashen Sun, and Hengliang Luo. 2016. When does relevance mean usefulness and user satisfaction in web search?. In *Proceedings of the 39th International ACM SIGIR conference on Research and Development in Information Retrieval*. 463–472.

[25] Bhaskar Mitra and Nick Craswell. 2017. Neural models for information retrieval. *arXiv preprint arXiv:1705.01509* (2017).

[26] Bhaskar Mitra, Nick Craswell, et al. 2018. An introduction to neural information retrieval. *Foundations and Trends® in Information Retrieval* 13, 1 (2018), 1–126.

[27] Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. 2016. Ms marco: A human-generated machine reading comprehension dataset. (2016).

[28] Leif E Peterson. 2009. K-nearest neighbor. *Scholarpedia* 4, 2 (2009), 1883.

[29] Zhen Qin, Rolf Jagerman, Kai Hui, Honglei Zhuang, Junru Wu, Le Yan, Jiaming Shen, Tianqi Liu, Jialu Liu, Donald Metzler, Xuanhui Wang, and Michael Bendersky. 2024. Large Language Models are Effective Text Rankers with Pairwise Ranking Prompting. In *Findings of the Association for Computational Linguistics: NAACL 2024*. 1504–1518.

[30] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research* 21, 140 (2020), 1–67.

[31] Carl Rasmussen and Zoubin Ghahramani. 2000. Occam's razor. *Advances in neural information processing systems* 13 (2000).

[32] Nils Reimers and Iryna Gurevych. 2020. Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 4512–4525.

[33] Stephen Robertson, Hugo Zaragoza, et al. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends® in Information Retrieval* 3, 4 (2009), 333–389.

[34] Artsiom Sauchuk, James Thorne, Alon Halevy, Nicola Tonellotto, and Fabrizio Silvestri. 2022. On the role of relevance in natural language processing tasks. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1785–1789.

[35] Yaodong Su, Yixiang Fang, Yingli Zhou, Quanqing Xu, and Chuanhui Yang. 2025. Clue-RAG: Towards Accurate and Cost-Efficient Graph-based RAG via Multi-Partite Graph and Query-Driven Iterative Retrieval. *arXiv preprint arXiv:2507.08445* (2025).

[36] Haoyu Wang, Ruirui Li, Haoming Jiang, Jinjin Tian, Zhengyang Wang, Chen Luo, Xianfeng Tang, Monica Xiao Cheng, Tuo Zhao, and Jing Gao. 2024. BlendFilter: Advancing Retrieval-Augmented Large Language Models via Query Generation Blending and Knowledge Filtering. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 1009–1025.

[37] Shuting Wang, Xin Yu, Mang Wang, Weipeng Chen, Yutao Zhu, and Zhicheng Dou. 2025. RichRAG: Crafting Rich Responses for Multi-faceted Queries in Retrieval-Augmented Generation. In *Proceedings of the 31st International Conference on Computational Linguistics*. 11317–11333.

[38] Yujing Wang, Hainan Zhang, Liang Pang, Binghui Guo, Hongwei Zheng, and Zhiming Zheng. 2025. MaFeRw: Query rewriting with multi-aspect feedbacks for retrieval-augmented large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 25434–25442.

[39] Yujing Wang, Hainan Zhang, Liang Pang, Yongxin Tong, Binghui Guo, Hongwei Zheng, and Zhiming Zheng. 2025. Learning to Erase Private Knowledge from Multi-Documents for Retrieval-Augmented Large Language Models. *arXiv preprint arXiv:2504.09910* (2025).

[40] Zhiruo Wang, Jun Araki, Zhengbao Jiang, Md Rizwan Parvez, and Graham Neubig. 2023. Learning to filter context for retrieval-augmented generation. *arXiv preprint arXiv:2311.08377* (2023).
[41] Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. 2024. C-Pack: Packed Resources For General Chinese Embeddings. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2024, Washington DC, USA, July 14-18, 2024*. 641–649.
[42] Fangyuan Xu, Weijia Shi, and Eunsol Choi. 2024. RECOMP: Improving retrieval-augmented LMs with context compression and selective augmentation. In *The Twelfth International Conference on Learning Representations*.
[43] Shicheng Xu, Liang Pang, Huawei Shen, Xueqi Cheng, and Tat-Seng Chua. 2024. Search-in-the-chain: Interactively enhancing large language models with search for knowledge-intensive tasks. In *Proceedings of the ACM Web Conference 2024*. 1362–1373.
[44] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
[45] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. 2369–2380.
[46] Qianchi Zhang, Hainan Zhang, Liang Pang, Hongwei Zheng, and Zhiming Zheng. 2024. Adacomp: Extractive context compression with adaptive predictor for retrieval-augmented large language models. *arXiv preprint arXiv:2409.01579* (2024).
[47] Qianchi Zhang, Hainan Zhang, Liang Pang, Hongwei Zheng, and Zhiming Zheng. 2026. Stable-RAG: Mitigating Retrieval-Permutation-Induced Hallucinations in Retrieval-Augmented Generation. *arXiv preprint arXiv:2601.02993* (2026).
[48] Xinping Zhao, Shouzheng Huang, Yan Zhong, Xinshuo Hu, Baotian Hu, and Min Zhang. 2025. Learning to Extract Rational Evidence via Reinforcement Learning for Retrieval-Augmented Generation. *arXiv preprint arXiv:2507.15586* (2025).
[49] Xinping Zhao, Dongfang Li, Yan Zhong, Boren Hu, Yibin Chen, Baotian Hu, and Min Zhang. 2024. SEER: Self-Aligned Evidence Extraction for Retrieval-Augmented Generation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 3027–3041.
[50] Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, and Zheyan Luo. 2024. LlamaFactory: Unified Efficient Fine-Tuning of 100+ Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*. 400–410.
[51] Yujia Zhou, Yan Liu, Xiaoxi Li, Jiajie Jin, Hongjin Qian, Zheng Liu, Chaozhuo Li, Zhicheng Dou, Tsung-Yi Ho, and Philip S Yu. 2024. Trustworthiness in retrieval-augmented generation systems: A survey. *arXiv preprint arXiv:2409.10102* (2024).
[52] Junda Zhu, Lingyong Yan, Haibo Shi, Dawei Yin, and Lei Sha. 2024. ATM: Adversarial Tuning Multi-agent System Makes a Robust Retrieval-Augmented Generator. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 10902–10919.
[53] Kun Zhu, Xiaocheng Feng, Xiyuan Du, Yuxuan Gu, Weijiang Yu, Haotian Wang, Qianglong Chen, Zheng Chu, Jingchang Chen, and Bing Qin. 2024. An Information Bottleneck Perspective for Effective Noise Filtering on Retrieval-Augmented Generation. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 1044–1069.
[54] Yutao Zhu, Huaying Yuan, Shuting Wang, Jiongnan Liu, Wenhan Liu, Chenlong Deng, Haonan Chen, Zheng Liu, Zhicheng Dou, and Ji-Rong Wen. 2023. Large language models for information retrieval: A survey. *arXiv preprint arXiv:2308.07107* (2023).

A Prompt

A.1 Prompt for the Clue Extractor

We present our prompt for the clue extractor in Table 9. The prompt is designed to guide the model in extracting the most informative sentences, which are most likely to contain the answer to the given question.

A.2 Prompt for the Adaptive Truncator

We present our prompt for the adaptive truncator in Table 10. The prompt is designed to guide the model in optimizing context truncation based on the complexity of the question and the quality of the retrieved documents, thereby improving the efficiency of the language model. Specifically, given a question and a ranked list of sentences, the model's task is to identify and retain the most relevant sentences while truncating those that are irrelevant to the question.

Table 9: Prompt for the Clue Extractor.

You are a highly skilled assistant specializing in extracting relevant information from provided documents. Your task is to identify and extract sentences from the documents as much as possible that are most directly useful for answering the given question. Rank the sentences in order of relevance, with the most relevant sentence listed first. Preface each sentence with its sequence number as follows:
Sentence 1:
.....
Sentence n:

Question:
`{Question}`

Documents:
`{Documents}`

Table 10: Prompt for the Adaptive Truncator.

You are a highly skilled assistant specializing in optimizing language model efficiency by truncating context based on question complexity and document quality. Given a question and a ranked list of sentences, identify and retain the most relevant ones while truncating the irrelevant sentences.

Question:
`{Question}`

Ranked List:
`{Ranked List}`

B More Experimental Settings

B.1 Datasets

We conduct experiments on three widely used QA datasets that cover both single-hop and multi-hop question-answering scenarios. Table 11 summarizes the key statistics of these datasets. Specifically, **NQ** (Natural Questions) and **TriviaQA** are representative single-hop datasets, where each question can typically be answered using information from a single passage retrieved from the corpus. These datasets primarily evaluate a model's ability to locate and extract

Table 11: Statistics for the datasets.

Dataset	Type	# Train	# Dev	# Test
NQ	single-hop	79.1k	8.7k	3.6k
TriviaQA	single-hop	78.7k	11.3k	8.8k
HotpotQA	multi-hop	88.9k	5.6k	5.6k

factual evidence efficiently. In contrast, **HotpotQA** is a challenging multi-hop dataset that requires integrating and reasoning over multiple pieces of evidence distributed across different documents to derive the final answer. This dataset is particularly useful for testing a model's reasoning and compositional understanding capabilities. Together, these datasets provide a comprehensive benchmark for evaluating both the retrieval quality and reasoning robustness of our proposed method under diverse task settings.

B.2 More Implementation Details

We use LLaMA3-8B-Instruct [4] [6], Qwen3-14B [5] [44], and LLaMA3.3-70B-Instruct [6] [6] as the generators, covering medium, large, and ultra-large LLMs. All of these models demonstrate strong performance across various tasks and exhibit high flexibility. To reduce computational cost and latency, our clue extractor and adaptive truncator are based on LLaMA3.2-3B-Instruct [7] [6]. We apply the LoRA method [9] for fine-tuning, an efficient low-rank adaptation technique that reduces the computational cost of parameter updates while maintaining model performance. LoRA is applied to both the clue extractor and adaptive truncator. We train the models on two NVIDIA RTX PRO 6000 GPUs for 12 epochs. The initial learning rate is set to 1×10^{-4}, and the batch size is 4. Gradient accumulation is employed to simulate larger effective batch sizes and improve training stability. The best model is selected based on validation set performance. During fine-tuning, the models are trained on the three QA datasets using a KNN-based sentence selection method. Data preprocessing is accelerated with 16 parallel workers per training epoch. The maximum input length is set to 4096 tokens to accommodate long-context information. For the clue reranker, we employ Sentence-BERT [32] with the distilbert-base-uncased model to generate high-quality sentence embeddings for computing sentence similarity. Training uses the Adam optimizer with a batch size of 64, a learning rate of 2×10^{-5}, and 1000 warm-up steps, and runs for 4 epochs.

C More Experimental Results on LLaMA3-70B

To highlight the generalization capability of our method, we introduce a new retriever, **Contriever-MS MARCO** [12, 27], which retrieves the Top-60 documents per query for the experiments below.

C.1 Full Content vs. Sentences w/ answers

Figure 5 presents the F1 performance of LLaMA3-8B-Instruct and LLaMA3.3-70B-Instruct under different Top-K retrieval settings,

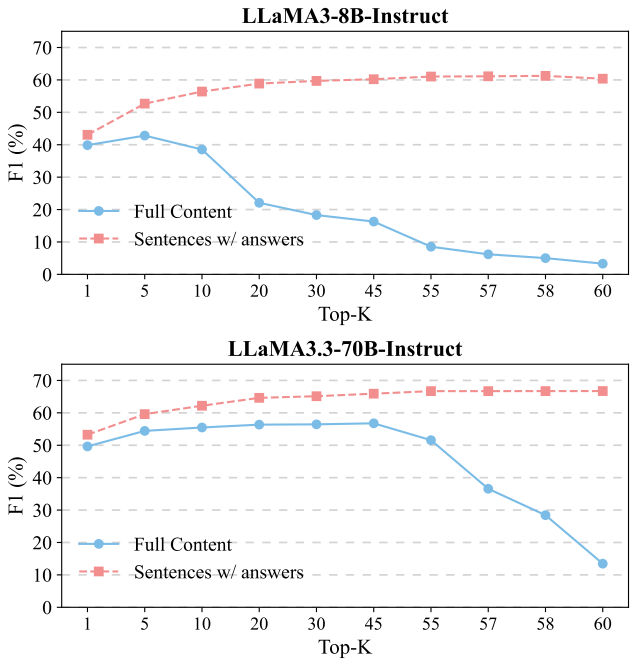


Figure 5: F1 performance of LLaMA3-8B-Instruct (Top) and LLaMA3.3-70B-Instruct (Bottom) across varying Top-K retrieval settings. The retriever is Contriever-MS MARCO. Performance is shown for two strategies: Full Content and Sentences w/ answers.

where documents are retrieved using the **Contriever-MS MARCO** retriever. Two input strategies are compared: ***Full Content*** and ***Sentences w/ answers***. For both model scales, performance initially improves as more documents are retrieved but later declines, with the larger model showing a delayed inflection point. Overall, the Sentences w/ answers strategy consistently surpasses the Full Content strategy across all Top-K configurations. Moreover, as the number of retrieved documents increases, the performance gain provided by Sentences w/ answers gradually diminishes and stabilizes, suggesting that this strategy offers robust and efficient context selection even under large-scale retrieval conditions.

C.2 Comparison with Baselines

Following the same experimental setting described in Section 5.1.4, we conduct experiments on NQ, TriviaQA, and HotpotQA. We use the **DPR** retriever to retrieve the Top-5 documents and adopt LLaMA3.3-70B-Instruct as the generator. All baselines and our method are conducted using the same test sets and retrieval corpus. The results are summarized in Table 12. Across all three datasets, our method consistently outperforms various baseline approaches. This demonstrates that our approach effectively selects the most informative context from retrieved documents, enhancing performance on downstream generation tasks. Moreover, the consistent performance gains highlight the stability and strong generalization ability of our method across different task distributions, making it applicable in a wide range of retrieval and generation scenarios.

[4] https://huggingface.co/meta-llama/Meta-Llama-3-8B-Instruct
[5] https://huggingface.co/Qwen/Qwen3-14B
[6] https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct
[7] https://huggingface.co/meta-llama/Llama-3.2-3B-Instruct

Table 12: Experimental results (%) on three benchmark datasets using LLaMA3.3-70B-Instruct as the generator. The retriever is DPR. The experimental setup follows Section 5.1.4. All baselines and our method are conducted using the same test sets and retrieval corpus.

Method	NQ			TriviaQA			HotpotQA		
	SubEM	F1	CR	SubEM	F1	CR	SubEM	F1	CR
LLaMA3.3-70B-Instruct									
Naive Generation	40.99	45.77	-	73.36	76.74	-	28.63	35.55	-
BM25	41.25	45.59	4.1×	69.44	69.02	4.3×	28.15	33.76	4.4×
Bge-reranker	43.94	50.22	4.1×	70.29	71.34	4.3×	29.13	35.83	4.5×
RECOMP-extr	43.60	50.03	11.97×	71.91	76.63	10.91×	29.61	36.25	8.33×
LongLLMLingua	48.39	53.67	4.56×	74.37	78.12	4.18×	31.03	38.42	4.45×
Selective-Context	50.14	55.79	2.6×	74.21	77.03	2.7×	30.88	37.93	2.7×
EXIT	47.25	51.83	14.16×	66.06	67.43	12.78×	27.48	32.25	**15.43×**
RECOMP-abs	47.39	51.89	11.12×	68.81	68.77	11.25×	27.95	34.75	7.91×
Refiner	49.22	54.02	10.97×	72.85	77.29	12.63×	30.42	37.12	7.65×
BottleNeck	49.53	54.44	14.32×	72.96	77.46	13.17×	30.75	37.76	13.21×
Ours ($\epsilon = 0$)	**50.35**	**56.21**	**17.19×**	**75.11**	**79.94**	**14.16×**	**31.36**	**38.75**	14.73×

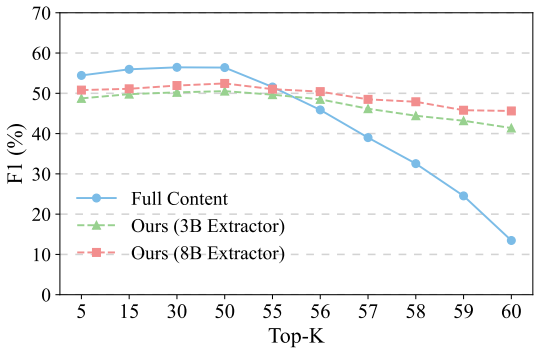


Figure 6: F1 scores on the NQ dataset under different Top-K retrieval settings for three strategies: Full Content, 3B Extractor, and 8B Extractor. The retriever is Contriever-MS MARCO. We use LLaMA3.3-70B-Instruct as the generator and F1 for evaluation.

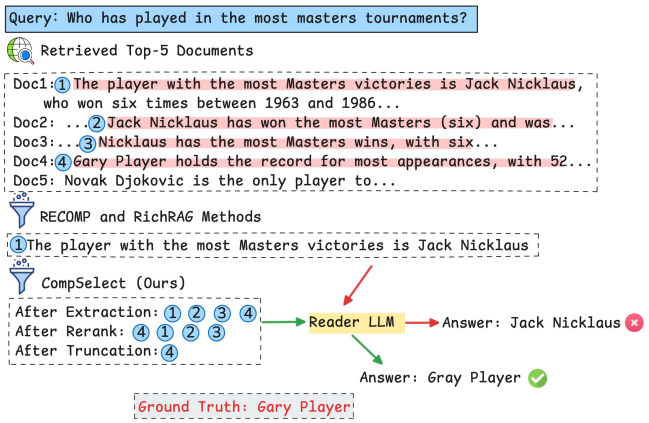


Figure 7: An illustration of the challenge in locating accurate answer clues. While baselines RECOMP and RichRAG select an incorrect clue from the first document, our method identifies the correct clue from the fourth via extraction, reranking, and truncation.

C.3 Comparison with Full Content Strategy

This section evaluates the generalization of our method across retrievers on the NQ dataset. We train with documents from the DPR retriever and test on the Contriever-MS MARCO retriever, simulating cross-retrieval shifts. Two extractors, LLaMA3.2-3B-Instruct (3B Extractor) and LLaMA3-8B-Instruct (8B Extractor), are compared against the Full Content baseline across Top-1 to Top-60 retrieved documents to analyze context-scale effects.

As shown in Figure 6, when the number of retrieved documents is small, our method performs slightly worse than the Full Content strategy. This is primarily because the LLaMA3.3-70B model itself has a sufficiently large context window and strong robustness, allowing it to effectively utilize information even under small-scale contexts. However, as the number of retrieved documents increases, the performance of the Full Content approach first rises and then declines, reflecting that the advantage of a large context window diminishes as document scale grows. In contrast, our method consistently outperforms the Full Content strategy when more documents are retrieved, indicating that selective truncation and compression can more effectively filter and leverage key information, thereby enhancing generation performance and stability under large-scale contexts. This pattern suggests that while smaller contexts may suffice for large LLMs, effective extraction of key sentences becomes increasingly important as more information is available.

Increasing the extractor size from 3B to 8B improves performance but increases latency, highlighting a trade-off between efficiency and quality. Larger extractors better select key information for large-scale contexts, whereas smaller extractors may be preferable under limited resources or real-time constraints.

D Case Study

As shown in Figure 7, our method highlights potential clues (red background), reranks them, and surfaces the correct answer clue at the top, while filtering out redundant ones to improve the information density of reasoning clues for RAG.